

1 RaftKV — A Dynamically Reconfigurable Distributed Key-Value Store

Project Proposal for UCS503P

Submitted to: Miss Anushka

Thapar Institute of Engineering and Technology

	Name	Department	Roll No.	Email
1	Kunwar Aran Srivastava	CSED	1024240003	ksrivastava_be24@thapar.edu
2	Yuvraj Kumar	CSED	1024240143	ykumar2_be24@thapar.edu
3	Anubhav Yadav	CSED	1024240104	ayadav6_be24@thapar.edu
4	Pratik Batish	CSED	1024240101	pbatish_be24@thapar.edu

1.1 1 Higher-order goal

This project aims to deepen understanding of distributed-systems theory — specifically consensus and fault-tolerant replication — by building a correct, from-scratch implementation of the Raft consensus algorithm inside a well-engineered Go codebase. While the principles of distributed consensus underpin critical production infrastructure (etcd, CockroachDB, Consul, Kubernetes coordination), the immediate engineering objective is to translate those principles into a testable, modular, and demonstrably correct key-value store that four students can fully understand, not merely assemble from opaque libraries.

RaftKV is explicitly **not** an attempt to build a production-grade or etcd-equivalent system. Its purpose is to demonstrate correct application of the Raft protocol — leader election, log replication, dynamic membership, crash recovery — inside a disciplined software project with automated testing, fault injection, CI/CD, and thorough documentation, built at approximately 10 hours per person per week over a 10–12 week schedule.

1.2 2 Time-to-value

- We will deliver meaningful, demonstrable increments at the end of each weekly milestone by sequencing work so that each week produces a testable, running artifact — not just code that compiles but cannot be exercised.
- We will prioritize correctness over feature breadth ensuring the core Raft consensus and KV operations are solid before any optional feature is attempted.
- We will use CI-backed automation from week 1 — every pull request triggers build, vet, lint, unit tests, and race detection (`go test -race`) — so that integration problems surface immediately, not during a final-week scramble.
- Success at each milestone is defined by what is **provably correct** (invariant-holding, race-free, crash-recoverable), not merely by what can be demoed on the happy path. The MVP (§9.1) is scoped to be achievable within the first 6 weeks, with subsequent weeks dedicated to membership, fault testing, and hardening.

1.3 3 Problem Statement

Single-node key-value stores have no fault tolerance: a crash or restart loses availability entirely, and there is no mechanism to survive machine failure without external replication. This is a fundamental limitation for any system that needs to remain available and consistent in the face of infrastructure failures.

Naively replicating writes to multiple nodes without a consensus protocol risks **split-brain** scenarios, **lost writes**, or **diverging replicas** under network partitions or node crashes. Common pain points include:

- **No crash tolerance:** A single node crash renders the entire store unavailable. Recovery requires manual intervention and risks data loss for any state that was not durably persisted.
- **Split-brain under partition:** Without a formal consensus mechanism, two network partitions may independently accept conflicting writes, leading to irreconcilable state divergence when the partition heals.
- **No dynamic membership:** Traditional static-cluster designs require the entire cluster to be stopped and reconfigured to add or remove a node — a significant operational burden and availability gap.
- **Unsafe crash recovery:** Without checksummed, fsync'd durable storage (write-ahead logging), a node that crashes mid-write may recover into an inconsistent state — partially applied operations, corrupted metadata, or a log that disagrees with what the node had previously acknowledged to its peers.

Why this matters now (contextual relevance): The Raft consensus algorithm (Ongaro & Ousterhout, 2014) was designed specifically to be more understandable than Paxos while providing equivalent safety guarantees. It is the foundation of widely deployed production systems (etcd, Consul, TiKV, CockroachDB). Building a correct implementation from scratch — and validating it under injected faults — is one of the most effective ways to internalize distributed-systems principles that are otherwise purely theoretical in a coursework setting.

1.4 4 Proposed Solution

1.4.1 4.1 Overview

Build a **from-scratch, dynamically reconfigurable, strongly consistent distributed key-value store** (RaftKV) with the following components:

- **Raft consensus core:** Leader election with randomized timeouts, log replication via AppendEntries RPCs, term-based voting, commit-index tracking with majority-quorum semantics, and persistent Raft metadata for crash recovery.
- **Replicated KV state machine:** A deterministic state machine that applies committed log entries as PUT/GET/DELETE operations against an in-memory key-value map, with snapshot and restore capabilities.
- **Dynamic cluster membership:** An operator-driven join protocol where new nodes enter as non-voting **learners**, catch up to the leader's committed log via AppendEntries (or InstallSnapshot for large gaps), and are promoted to **voters** only once sufficiently caught up — ensuring that adding a node never endangers cluster availability.
- **Durable storage engine:** A write-ahead log (WAL) with checksummed, fsync'd records for Raft metadata and log entries, plus periodic snapshots of the KV state for log compaction and efficient new-node catch-up.
- **Client-facing API and CLI:** An HTTP/JSON API for PUT/GET/DELETE operations and cluster management, plus a `raftctl` CLI for operator workflows (cluster status, join, remove).
- **Fault-injection and testing framework:** A scriptable harness for crash, partition, delay, and message-drop injection, validating Raft safety invariants under realistic failure scenarios.

1.4.2 4.2 Core workflow

1. **Client submits a request** (PUT, GET, or DELETE) via HTTP to any node in the cluster.
2. **Leader check:** If the receiving node is not the current leader, it responds with a structured redirect (409 Conflict + leaderHint JSON body) pointing the client to the leader's address. The client (or raftctl CLI) retries against the indicated leader.
3. **Leader proposes the command:** The leader wraps the client's operation into a Raft log entry (with the current term and next log index) and appends it to its local log.
4. **Log replication:** The leader sends the new entry to all followers and learners via gRPC AppendEntries RPCs. Each follower validates the entry's consistency (prevLogIndex/prevLogTerm check), appends it to its own WAL (fsync'd before responding), and acknowledges.
5. **Quorum commit:** Once the leader receives acknowledgments from a **majority of current voting members** (not including learners), it advances its commit index to include the new entry. Entries are committed in log order.
6. **State-machine application:** Each node (leader and followers) applies committed entries to its KV state machine in index order. The leader then responds to the client with the operation's result.
7. **For membership changes:** A new node sends a JoinCluster RPC to any existing node (which redirects to the leader if necessary). The leader appends a configuration-change entry to the log to add the node as a learner, replicates it, and once committed begins replicating to the learner. After the learner's matchIndex reaches within a configurable threshold of the leader's commitIndex, the leader proposes a second config-change entry to promote the learner to a voter, committed under the old quorum.

1.4.3 4.3 Operational constraints for an educational setting

- **Single-key operations only:** No multi-key transactions, no sharding, no partitioned keyspaces. The focus is on consensus correctness, not database feature richness.
- **Trusted network assumption:** No TLS, no authentication, no authorization in the mandatory scope. The system assumes a private Docker network or trusted LAN. This is documented as a known limitation, not presented as production-safe.
- **Correctness over performance:** No throughput SLAs or competitive benchmarks. A sane target is single-digit to low-tens-of-millisecond local commit latency on a 3–5 node Docker cluster — a sanity bound, not a published guarantee.
- **Local/Docker deployment only:** Docker Compose for multi-node clusters. Kubernetes manifests, cloud deployment, and multi-host orchestration are explicitly out of scope.
- **One membership change at a time:** Concurrent multi-node joins/removals are disallowed, avoiding the complexity of joint consensus. This matches the operator-driven join philosophy and simplifies correctness reasoning.

1.5 5 Solution Approach

This is an engineering course project. The focus is on correct implementation of a well-understood distributed-systems algorithm (Raft) and professional software engineering practices, not on novel algorithm research.

1.5.1 5.1 Raft consensus implementation

The Raft core module (`internal/raft`) implements the full Raft state machine with three roles — **Follower**, **Candidate**, and **Leader** — plus a learner membership attribute layered on the Follower state (learners run the same replication logic but never vote and are never counted toward quorum).

Leader election: Nodes start as followers. If a follower receives no heartbeat within a randomized election timeout (150–300ms range, per Raft paper guidance), it transitions to candidate, increments its term, votes for itself, and sends `RequestVote` RPCs to all peers. A candidate wins if it receives votes from a majority of voters; it steps down to follower if it discovers a higher term. At most one leader can exist per term (Election Safety invariant S1), enforced by the persisted `votedFor` field — a node votes at most once per term.

Log replication: The leader appends client commands as log entries and replicates them via `AppendEntries` RPCs. Each RPC includes `prevLogIndex` and `prevLogTerm` for the Log Matching consistency check (invariant S3): a follower rejects an append if its log doesn’t match at the specified position, and the leader retries with earlier entries until consistency is established. Conflict resolution truncates divergent follower entries and replaces them with the leader’s entries — the leader never overwrites its own log (Leader Append-Only, invariant S2).

Commit and apply: An entry is committed once a majority of current voters have acknowledged it. The leader advances its `commitIndex` and communicates it to followers via subsequent `AppendEntries` RPCs. Each node applies committed entries to the KV state machine in strict index order via a deterministic `Apply(entry)` interface — ensuring State Machine Safety (invariant S5): every replica applies the same entry at every index.

Persistence: `currentTerm`, `votedFor`, and all log entries are durably persisted (WAL + fsync) **before** any response that implies their durability is sent. This is the foundation of crash recovery: a restarted node reloads its persisted state and rejoins as a follower, catching up via normal `AppendEntries` or `InstallSnapshot`.

1.5.2 5.2 System architecture

RaftKV is decomposed into nine modules with strict dependency rules enforced by code review:

Module	Package	Responsibility
Raft Core	<code>internal/raft</code>	Election, replication, commit, persistence
Storage Engine	<code>internal/storage</code>	WAL, snapshots, crash recovery, compaction
Membership Manager	<code>internal/membership</code>	Join, learner tracking, promotion, removal
Transport Layer	<code>internal/transport</code>	gRPC client/server, Protobuf serialization
KV State Machine	<code>internal/state</code>	Deterministic Apply for PUT/GET/DELETE
Client API	<code>internal/api</code>	HTTP handlers, leader-redirect logic
CLI	<code>cmd/raftctl</code>	Thin wrapper over HTTP API
Fault Injection	<code>internal/fault + tests/</code>	Crash/partition/delay injection hooks
Observability	<code>internal/observability</code>	Structured logging, Prometheus metrics

Coupling rules: The Raft core defines `Transport` and `StateMachine` interfaces (dependency inversion) — it never imports transport, API, or membership packages directly. The KV state machine has no knowledge of Raft, networking, or storage — it only implements `Apply(entry) → result` plus `Snapshot()/Restore()`. The transport layer implements the Raft-defined `Transport` interface and imports only generated Protobuf

code, never Raft internals. These rules prevent a “big ball of mud” architecture and ensure every module is testable in isolation.

1.5.3 5.3 Dynamic membership

Configuration changes (add learner, promote to voter, remove member) are represented as **special entries in the same replicated Raft log** used for KV commands, committed via the same quorum mechanism. This reuses all existing replication/commit machinery and avoids inventing a second consensus path.

Join protocol: 1. New node sends `JoinCluster(nodeID, address)` to any existing node, which redirects to the leader. 2. Leader validates: no change already in progress, `nodeID` not already a member. 3. Leader appends a config-change entry (`CONFIG_ADD_LEARNER`) to the log and replicates it. 4. Once committed, the new node is a known learner; the leader begins replicating to it. 5. If the learner’s log is empty or far behind, the leader sends its latest snapshot (`InstallSnapshot`) followed by `AppendEntries` for the remaining tail. 6. Once the learner’s `matchIndex` is within a configurable threshold (e.g., 10 entries) of the leader’s `commitIndex`, the leader proposes a promotion entry (`CONFIG_PROMOTE_VOTER`), committed under the **old** quorum — the newly promoted node does not vote on its own promotion.

Node removal: An operator issues `raftctl cluster remove <nodeID>`. The leader validates the removal won’t drop below minimum quorum size, proposes a `CONFIG_REMOVE` entry, and commits it under the old voter set. Self-removal of the current leader is disallowed in this scope.

Safety invariant (S6): A learner never counts toward quorum. A configuration change only takes effect once committed. At most one membership change is in flight at a time.

1.5.4 5.4 Storage and durability

Write-ahead log (WAL): All Raft metadata and log entries are persisted as checksummed, append-only records in a single WAL file per node. Each record contains a length field (for torn-write detection), a CRC checksum, a record type (`STATE` or `LOG_ENTRY`), and the serialized payload. The critical write-path invariant: `append → fsync → return success`. No response implying durability (vote granted, `AppendEntries` ack, client write acknowledged) may be sent before the relevant `fsync` completes.

Crash recovery: On startup, the WAL is sequentially replayed. On the first checksum failure or truncated frame, reading stops — everything before that point is the recovered log. This correctly handles a crash mid-write of the last record (a classic, expected WAL failure mode). Recovery is idempotent: a crash during recovery itself is safe because recovery reads persisted, checksummed state without mutating it until fully validated.

Snapshots and compaction: The KV state machine is periodically snapshotted (triggered every N committed entries, configurable). Each snapshot records `lastIncludedIndex`, `lastIncludedTerm`, the serialized KV map, and a checksum. After a snapshot is durably written, WAL entries with index `lastIncludedIndex` are discarded (compaction), preventing unbounded WAL growth. Snapshot transfer to lagging or new nodes uses a single-message `InstallSnapshot` RPC — a known scalability limit for very large state, documented as a limitation with chunked/streaming transfer deferred to optional scope.

1.5.5 5.5 CI/CD and delivery pipeline

GitHub Actions (`.github/workflows/ci.yml`) runs on every pull request and push to main:

1. `go build ./...` — compilation check
2. `go vet ./...` — static analysis
3. `golangci-lint run` — code quality
4. `go test ./...` — unit and integration tests
5. `go test -race ./...` — data-race detection (non-negotiable for a Go concurrency project)

Docker image build as a CI job produces a `raftkv` container image, validated by starting a Docker Compose cluster and hitting `/cluster/status` — a cheap smoke test that catches “it doesn’t even boot” regressions.

Git workflow: `main` is always green. Feature branches named `area/short-description` (e.g., `raft/leader-election`). Pull requests required for all merges with minimum 1 reviewer; 2 reviewers recommended for `internal/raft`

and **internal/membership** changes. Commits reference which Raft safety invariants (S1–S7) a change touches, if any.

Docker Compose provides a configurable N-node local cluster with per-node named volumes for durable storage. Scripts in `scripts/` demonstrate dynamic node join against a running cluster.

1.6 6 Evaluation Criterion

We define success using measurable properties tied to the core correctness and engineering quality of the system. Unlike a web application where user-facing latency metrics are primary, a consensus system’s value proposition is **safety under failure** — the metrics reflect this.

1.6.1 6.1 Primary evaluation metric

Raft Safety Invariant Compliance: All seven safety invariants (S1–S7) hold under fault injection across all test scenarios.

ID	Invariant	How Measured
S1	Election Safety — at most one leader per term	Property test: across hundreds of randomized election runs, assert no two nodes report Leader for the same term
S2	Leader Append-Only — leader never overwrites its own log	Unit test + integration assertion on leader’s log after conflict resolution
S3	Log Matching — same index+term implies identical prefix	Integration test: compare logs across all nodes after replication under churn
S4	Leader Completeness — committed entries present in all future leaders’ logs	Fault test: kill leader after commit, verify new leader has the entry
S5	State Machine Safety — no two nodes apply different entries at the same index	Integration test: compare applied KV state across all nodes after writes
S6	Membership-Change Safety — config change only effective once committed; learner never in quorum	Membership tests: assert quorum size during and after join/promotion
S7	No commit without quorum — entry committed only after majority voter ack	Partition test: write to minority side, assert it never commits

- **Target:** Zero invariant violations across the full fault-injection test suite.
- **Attribution:** Each invariant is checked by assertions embedded in the test harness, producing pass/fail results traceable to specific test scenarios.

1.6.2 6.2 Secondary evaluation metrics

- **Race-detector cleanliness:** `go test -race ./...` passes with **zero** data races, consistently (not “usually”). This is a binary pass/fail, non-negotiable for a Go concurrency project.
- **Crash-recovery correctness:** A node that crashes and restarts converges to the cluster’s committed state without data loss or corruption, validated by WAL checksum verification and post-recovery state comparison.

- **Fault-injection coverage:** The test suite covers, at minimum: leader crash → re-election, follower crash → continued commits, network partition → minority can't commit while majority continues, node restart → catch-up convergence, join during concurrent writes → learner catches up.
- **Join protocol completeness:** A new node can join a running cluster, catch up (via AppendEntries or InstallSnapshot), be promoted to voter, and participate in subsequent elections and commits.
- **Test tier coverage:** Unit, integration (in-process multi-node), integration (Docker Compose), fault-injection, and membership test tiers all contain tests — no tier is left empty.

1.6.3 6.3 Validation plan

Validation is structured in five tiers, each exercising different failure surfaces:

Tier	Scope	Examples
Unit	Single-module logic in isolation	Election vote-granting rules, log-matching/conflict resolution, quorum-size calculation for various N, WAL checksum/recovery, snapshot serialize/restore
Integration (in-process)	Multiple Raft instances in one test binary	3-node and 5-node election; replication under no faults; PUT/GET/DELETE end-to-end through API layer
Integration (Docker)	Full multi-container cluster	Docker Compose bring-up; join a real container; <code>docker kill</code> a leader and observe re-election via HTTP status polling
Fault injection	Injected failures via transport interceptor	Leader crash, follower crash, multiple failures below quorum (writes refused), network partition (minority never commits), delayed/dropped packets
Membership	Dynamic cluster changes under stress	Join, join during concurrent writes, learner catch-up, promotion, restart during catch-up, duplicate/late join retries

Property-style correctness checks run across many randomized fault-injection scenarios, asserting S1–S7 invariants. These run in CI as an extended suite (nightly if runtime exceeds per-PR budget).

Required CI commands (must pass on every PR):

```
go test ./...
go test -race ./...
```

1.7 7 Scalability

The system scales within the bounded scope of an educational project:

1. **Theoretically:**

- Configurable N-node clusters (quorum adapts automatically).
- Dynamic node addition via explicit join protocol.
- Snapshot-based catch-up avoids full log replay.
- Stateless client API (HTTP/JSON).

2. Practically:

- Docker Compose for local containerized deployment.
- CI/CD pipeline for automated testing.
- Zero external dependencies beyond Go, gRPC, and Docker.

1.8 8 Engine availability heuristic

A mature set of “engines” (tooling, frameworks, and services) is available to implement this project, enabling the team to focus on consensus correctness and engineering discipline rather than building or debugging infrastructure:

Engine	Choice	Rationale
Language	Go 1.22+	Excellent concurrency primitives (goroutines, channels), built-in race detector (<code>-race</code> flag), strong standard library, native cross-compilation, and the dominant language for Raft implementations (etcd, Consul, HashiCorp Raft)
Node-to-node transport	gRPC + Protocol Buffers	Strongly typed RPC contracts, built-in streaming (useful for snapshot transfer), mature Go tooling (<code>protoc-gen-go-grpc</code>), and the mandated technology requirement
Client API	HTTP/JSON (net/http)	Universal client compatibility, simple to test with curl/Postman, no client-side library dependency
Durable storage	Custom WAL + filesystem	No external database dependency; the WAL is itself a core deliverable demonstrating durable, checksummed storage
Containerization	Docker + Docker Compose	Industry-standard, widely available, supports named volumes for persistence and network isolation for partition simulation
CI/CD	GitHub Actions	Free for public repos, native Go support, integrated with PR workflow
Observability	Structured JSON logging + Prometheus text format	Lightweight, no external backend required; Prometheus exposition on <code>/metrics</code>
Testing	Go’s <code>testing</code> package + race detector	Built-in, zero-dependency, integrates with CI; race detector is essential for concurrent code

Engine	Choice	Rationale
Documentation	Markdown + Mermaid diagrams + ADRs	Renderable in GitHub, version-controlled alongside code, low-friction to maintain

These are mature, well-documented components. The team will use them to focus on workflow design, Raft correctness, and engineering measurement rather than inventing infrastructure.

1.9 9 Project scope and deliverables

1.9.1 9.1 Initial deliverable

Initially, we will focus on building a correct, configurable N-node cluster with foundational Raft consensus. This includes: - Implementing correct leader election, term management, and log replication. - Supporting PUT/GET/DELETE operations via HTTP, with automatic leader-redirect for non-leader writes. - Ensuring crash recovery through a durable Write-Ahead Log (WAL) that is checksummed and fsync'd. - Adding basic snapshotting and log compaction. - Providing a gRPC transport layer for node-to-node RPCs. - Establishing core documentation (Software Design Document, API docs, diagrams). - Implementing robust CI/CD with automated build, test, and race detection on every pull request.

MVP feature set: - Configurable N-node cluster with correct leader election and log replication - PUT/GET/DELETE via HTTP with leader-redirect for non-leader writes - Durable WAL + crash recovery (checksummed, fsync'd) - Basic snapshotting + log compaction - gRPC transport layer for all node-to-node RPCs - `go test -race ./... clean` - CI running build/vet/test/race on every PR - Core documentation: ratified SDD, API docs, Mermaid diagrams

Team responsibility (primary ownership, not isolated silos):

Area	Primary Owner	Cross-review
Raft Core (election, replication, commit)	Person 1	Person 2 (storage), Person 4 (fault tests)
Storage + Membership (WAL, snapshots, join)	Person 2	Person 1 (Raft integration), Person 4 (membership tests)
Networking + API (gRPC, HTTP, CLI, redirect)	Person 3	Person 1 (transport contract), Person 2 (join wiring)
Testing + Reliability + DevOps (fault injection, CI/CD, Docker)	Person 4	All (each module's tests reviewed by its owner)

1.9.2 9.2 Subsequent deliverables

Following the MVP, we will enhance the cluster's operational capabilities and reliability: - Implementing dynamic membership with a learner role, join protocol, and catch-up mechanism. - Adding snapshot transfer for lagging nodes and learner promotion. - Hardening crash-recovery and running extensive fault-injection simulations (network partitions, race detection). - Polishing observability with metrics endpoints, structured logging, and basic health checks. - Finalizing testing, architecture documentation, and preparing demo scenarios.

Stretch goals (in priority order, only after MVP is stable): 1. Node removal — safe voter removal, timeboxed 2. Read-index / leader-lease linearizable reads — closes the known stale-read gap 3. Chunked/streaming snapshot transfer 4. Deeper metrics: commit latency histograms, catch-up lag gauges 5. Performance benchmarking suite + write-up 6. Watches, TTL/leases, CAS, simple transactions

Hard rule: None of the stretch goals may begin until the MVP items are implemented, tested, and documented.

1.10 10 Risks and mitigations

#	Risk	Likelihood	Impact	Mitigation
R1	Log replication/conflict resolution (week 4) takes longer than planned — historically the single hardest correctness-critical chunk of any Raft implementation	High	High	Explicit buffer in roadmap; defer non-critical polish (metrics, CLI niceties) to protect this week. Do not allow week 4 work to be interrupted by scope additions.
R2	Membership protocol (weeks 7–8) is underestimated — second hardest chunk; depends on Raft core being fully solid	High	High	Hard sequencing dependency: do not start membership work until election + replication tests are green. This is not parallelizable with incomplete Raft core.
R3	Fault-injection harness is built too late to validate earlier weeks’ correctness claims	Medium	High	Build a minimal harness (process kill + simple message drop) as early as week 3–4, even before full Raft correctness is done. This validates the harness itself early and gives Person 4 real integration points.
R4	Under-investment in tests relative to “getting it to demo” — a classic distributed-systems trap where the happy path works but failure paths are silently broken	Medium	High	Definition of Done explicitly requires failure tests, not just unit tests, for every Raft/membership feature. Code review checklist references safety invariants (S1–S7).

#	Risk	Likelihood	Impact	Mitigation
R5	Scope creep toward “make it more like etcd” — a common temptation once the core works and feels rewarding	Medium	Medium	Strict scope limitations (§4 of SDD) and prioritization (§9) re-read before adding anything not already listed. No feature additions without team consensus and schedule impact assessment.
R6	Uneven distributed-systems background across the 4-person team — may cause uneven module quality	Medium	Medium	Weekly integration sessions and mandatory cross-review on Raft/membership PRs surface gaps early (week 2–3), not at week 11. All team members must be able to explain the full system end-to-end.
R7	Underestimating documentation/report-writing time in the final weeks	Medium	Medium	Week 12 is explicitly budgeted for this. ADRs are written incrementally throughout the project (as part of each PR), not batched at the end.

Schedule compression policy: If the schedule must be compressed to 8–9 weeks, cut in this order: (1) node removal entirely, (2) read-index/linearizable reads, (3) metrics beyond 5 gauges, (4) fault-test breadth (keep only leader-crash + partition, the two highest-value correctness demonstrations). **Never cut:** election/replication/commit correctness and its tests, WAL durability tests, the race-detector requirement, or the join/learner/promotion happy path — cutting any of these removes the project’s core distributed-systems value entirely.

1.11 11 Summary

This project proposes a from-scratch, dynamically reconfigurable, strongly consistent distributed key-value store built on the Raft consensus algorithm in Go. It emphasizes time-to-value through iterative milestones, prioritizing a well-scoped MVP (leader election, log replication, KV operations, WAL persistence, crash recovery) followed by dynamic membership and rigorous fault injection. By implementing CI/CD from day 1, the system supports safe integration and mitigates concurrency risks. Success is measured by Raft safety invariant compliance (S1–S7) under failure scenarios rather than happy-path demos. Bounded in scope to single-key operations without sharding or auth, the project focuses on engineering correctness and

distributed-systems understanding within an achievable timeframe, delivering a deliberately bounded but real, demonstrably correct consensus implementation.